CODE WALKTHROUGH

From a raw ASTM/HL7 byte stream to a row in lab_results

Every excerpt below is read out of the repository when this PDF is built, so the document always matches the code that ships. Bengali project overview: docs/PROJECT_GUIDE_BN.pdf.

MAIN.PY

1 · Entry point and wiring

`main.py` loads `config.yaml`, decides between Oracle and the in-memory store, then starts one listener per enabled analyzer. Serial analyzers get a thread, TCP analyzers get an asyncio task.

```
def start_listeners(config: Dict[str, Any], store: Any) -> List[Any]:
    started: List[Any] = []
    loop = asyncio.get_running_loop()

    for analyzer in config.get("analyzers", []) or []:
        if not analyzer.get("enabled", True):
            logger.info("skipped (disabled): %s", analyzer.get("name"))
            continue

        engine = build_engine(config, store, analyzer)
        conn_type = str(analyzer.get("connection_type", "TCP")).upper()

        if conn_type == "SERIAL":
            listener = SerialListener(analyzer, engine, loop=loop)
            listener.start()
        else:
            listener = TCPListener(analyzer, engine)
            asyncio.create_task(listener.start(), name=analyzer.get("name"))

        started.append(listener)

    return started


# ---------------------------------------------------------------------- main
async def run(config_path: str, dry_run: bool) -> None:
```

CORE/PARSER_FACTORY.PY

2 · Protocol sniffing

Analyzers get misconfigured more often than people admit. The sniffer looks at the first segment of the payload; `parse_message` retries with the other parser if the configured one returns nothing, and logs that the config is wrong.

```
def sniff_protocol(raw: str) -> str:
    """Return 'HL7' or 'ASTM' for a raw message payload."""
    if not raw:
        return "ASTM"
    text = raw.replace("\x0b", "").replace("\x1c", "").lstrip()
    first = text.split("\r")[0].split("\n")[0].strip()
    if any(first.upper().startswith(seg + "|") for seg in HL7_SEGMENTS):
        return "HL7"
    if first[:1].upper() == "H":
        return "ASTM"
    if "|" in text and any(f"|{seg}|" in text for seg in HL7_SEGMENTS):
        return "HL7"
    return "ASTM"


def build_parser(protocol: str, test_code_map: Optional[Dict[str, str]] = None):
    if protocol.upper() == "HL7":
        return HL7Parser(test_code_map)
```

```
def parse_message(
    raw: str,
    protocol: str = "AUTO",
    analyzer_name: str = "UNKNOWN",
    test_code_map: Optional[Dict[str, str]] = None,
) -> List[Dict[str, Any]]:
    """Parse with the configured protocol, falling back to sniffing."""
    chosen = protocol.upper()
    if chosen in ("AUTO", ""):
        chosen = sniff_protocol(raw)

    parser = build_parser(chosen, test_code_map)
    results = parser.parse(raw, analyzer_name)

    if not results and chosen != "AUTO":
        other = "HL7" if chosen == "ASTM" else "ASTM"
        retry = build_parser(other, test_code_map).parse(raw, analyzer_name)
        if retry:
            logger.warning(
                "%s sent %s but parsed as %s — check analyzer config",
                analyzer_name, chosen, other,
            )
            return retry

    return results
```

PARSERS/ASTM_PARSER.PY

3 · ASTM record parsing

Records are split on CR (LF and CRLF tolerated), then on |. Patient and order records are read once and attached to every result record, so each row is self-contained for the database.

```
def _split_records(raw: str) -> List[List[str]]:
    """Split a message into records, each record into fields."""
    records: List[List[str]] = []
    for line in _clean(raw).split(CR):
        line = line.strip()
        if not line:
            continue
        records.append([field.strip() for field in line.split("|")])
    return records


def _field(fields: List[str], index: int, default: str = "") -> str:
```

```
    def parse(self, raw_message: str, analyzer_name: str = "UNKNOWN") -> List[Dict[str, Any]]:
        records = _split_records(raw_message)
        if not records:
            logger.warning("empty ASTM message from %s", analyzer_name)
            return []

        header = self._header(records)
        sample = self._order(records)
        patient = self._patient(records)

        results: List[Dict[str, Any]] = []
        for fields in records:
            if not fields or fields[0].upper() != "R":
                continue
            result = self._result_record(fields, analyzer_name, header, sample, patient, raw_mes
    sage)
            if result:
                results.append(result)

        logger.info("%s: parsed %d result(s) from %s", self.name, len(results), analyzer_name)
        return results

    # ------------------------------------------------------------------ private
    def _header(self, records: List[List[str]]) -> Dict[str, str]:
        for fields in records:
```

PARSERS/HL7_PARSER.PY

4 · HL7 OBX parsing

Segments are grouped by name and the parser anchors on `OBX`. Field indexes follow the HL7 numbering (OBX-2 value type, OBX-3 identifier, OBX-5 value, OBX-8 flag).

```
    def _obx(
        self,
        fields: List[str],
        analyzer_name: str,
        context: Dict[str, Any],
        raw_message: str,
    ) -> Optional[Dict[str, Any]]:
        identifier = self._at(fields, 3)          # CODE^TEXT^LOINC
        value_type = self._at(fields, 2) or "NM"
        raw_value = self._at(fields, 5)

        code = identifier.split("^")[0].strip()
        if not code:
            return None
        code = self.test_code_map.get(code.upper(), code)

        abnormal = (self._at(fields, 8) or "N").upper()
        status = self._at(fields, 11) or "F"
        observed_at = _parse_hl7_datetime(self._at(fields, 14))

        return {
            "analyzer_name": analyzer_name,
            "sample_id": context.get("sample_id") or context.get("patient_id", ""),
            "patient_id": context.get("patient_id", ""),
            "patient_name": context.get("patient_name", ""),
            "test_code": code,
            "result_value": _to_float(raw_value) if value_type.upper() != "ST" else None,
            "result_text": raw_value,
            "result_unit": self._at(fields, 6),
            "ref_range": self._at(fields, 7),
            "abnormal_flag": abnormal,
            "result_status": status,
            "result_datetime": observed_at or context.get("sent_at") or datetime.now().strftime(
    "%Y-%m-%d %H:%M:%S"),
            "raw_message": raw_message,
```

CORE/VALIDATOR.PY

5 · Validation rules

One place decides what a valid result is: required keys, unit/flag normalisation, critical detection (`C`, `HH`, `LL`) and the control-sample filter.

```
    def normalise(self, result: Dict[str, Any]) -> Dict[str, Any]:
        out = dict(result)
        out["sample_id"] = str(out.get("sample_id") or "").strip()
        out["patient_id"] = str(out.get("patient_id") or "").strip()
        out["patient_name"] = str(out.get("patient_name") or "").strip()
        out["test_code"] = str(out.get("test_code") or "").strip().upper()
        out["analyzer_name"] = str(out.get("analyzer_name") or "").strip()
        out["result_unit"] = str(out.get("result_unit") or "").strip()
        out["ref_range"] = str(out.get("ref_range") or "").strip()
        out["result_text"] = str(out.get("result_text") or "").strip()

        flag = str(out.get("abnormal_flag") or "").strip().upper()
        out["abnormal_flag"] = FLAG_ALIASES.get(flag, flag or "N")

        out["is_critical"] = "Y" if out["abnormal_flag"] in CRITICAL_FLAGS else "N"
        out["is_abnormal"] = "Y" if out["abnormal_flag"] not in {"N", ""} else "N"

        if not out.get("result_datetime"):
            out["result_datetime"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # numeric results may still arrive as strings from a custom parser
        if out.get("result_value") is not None:
            try:
                out["result_value"] = float(str(out["result_value"]).replace(",", "."))
            except (TypeError, ValueError):
                out["result_value"] = None

        out["protocol"] = str(out.get("protocol") or "").upper()
        return out
```

```
    def _problem(self, result: Dict[str, Any]) -> Optional[str]:
        for key in REQUIRED_FIELDS:
            if not result.get(key):
                return f"missing {key}"
        if result.get("result_value") is None and not result.get("result_text"):
            return "no value"
        if self.ignore_prefixes and result["sample_id"].upper().startswith(self.ignore_prefixes)
    :
            return f"control sample ({result['sample_id']})"
        if self.known_test_codes and result["test_code"] not in self.known_test_codes:
            return f"unknown test code {result['test_code']}"
        return None
```

CORE/ENGINE.PY

6 · The pipeline itself

Order matters: raw payload first, parse second, insert third, and only then build the ACK. If any step fails, the raw message is already safe and the analyzer gets a NAK instead of a silent drop.

```
    def process(self, raw: str, analyzer: Dict[str, Any], peer: str = "n/a") -> Dict[str, Any]:
        """Synchronous core — easy to unit test, easy to call from a thread."""
        analyzer_name = analyzer.get("name", "UNKNOWN")
        configured = str(analyzer.get("protocol", "AUTO")).upper()
        started = datetime.now()

        message_id = self.store.log_raw_message(analyzer_name, configured, raw, peer)
        self.counters["messages"] += 1
```

```
        entry: Dict[str, Any] = {
            "message_id": message_id,
            "analyzer_name": analyzer_name,
            "peer": peer,
            "received_at": started.strftime("%Y-%m-%d %H:%M:%S"),
            "configured_protocol": configured,
            "raw": raw,
        }

        try:
            protocol = sniff_protocol(raw)
            entry["detected_protocol"] = protocol

            code_map = dict(self.test_code_map)
            code_map.update(self.analyzer_code_maps.get(analyzer_name, {}))
            parsed = parse_message(
                raw,
                protocol=configured if configured != "AUTO" else "AUTO",
                analyzer_name=analyzer_name,
                test_code_map=code_map,
            )
            entry["parsed_count"] = len(parsed)

            if not parsed:
                entry.update(status="ERROR", note="no results parsed", stored=0)
                self.counters["errors"] += 1
                self.store.mark_message(message_id, "ERROR", "no results parsed")
                self._record(entry)
                return self._response(False, entry, "no results parsed")

            validated = self.validator.validate(parsed)
            entry["valid_count"] = len(validated)
            if not validated and getattr(self.validator, "last_ignored", None):
                sample = getattr(self.validator, "last_ignored")[0].split("(")[-1].rstrip(")")
                note = f"control sample {sample} ignored (not patient data)"
                entry.update(status="IGNORED", note=note, stored=0)
                self.store.mark_message(message_id, "IGNORED", note)
                self.store.insert_audit("IGNORED", analyzer_name, note)
                self._record(entry)
                return self._response(True, entry, note)

            if not validated:
                entry.update(status="ERROR", note="all results failed validation", stored=0)
                self.counters["errors"] += 1
                self.store.mark_message(message_id, "ERROR", "validation failed")
                self._record(entry)
                return self._response(False, entry, "validation failed")

            stored = self.store.insert_results(validated, raw)
            duplicates = len(validated) - stored
            self.counters["results"] += stored
            self.counters["duplicates"] += duplicates
```

DATABASE/DB_WRITER.PY

7 · Oracle writer

Two tables: `lis_raw_messages` (verbatim archive, with status) and `lab_results` (one row per test, de-duplicated). The duplicate check is cheap and keeps the analyzer's re-transmissions out of the report.

```
    def is_duplicate(self, result: Dict[str, Any]) -> bool:
        sql = f"""
            SELECT COUNT(*)
              FROM {self.schema}.lab_results
             WHERE sample_id     = :sample_id
               AND test_code     = :test_code
               AND analyzer_name = :analyzer_name
               AND result_value  = :result_value
        """
        with self.db.cursor() as cur:
            cur.execute(
                sql,
                {
                    "sample_id": result["sample_id"],
                    "test_code": result["test_code"],
                    "analyzer_name": result["analyzer_name"],
                    "result_value": result.get("result_value"),
                },
            )
            (count,) = cur.fetchone()
```

```
    def insert_results(self, results: Iterable[Dict[str, Any]], raw_message: str = "") -> int:
        """Insert results, skipping duplicates. Returns how many were stored."""
        stored = 0
        for result in results:
            try:
                if self.is_duplicate(result):
                    logger.warning(
                        "duplicate skipped: %s / %s",
                        result["sample_id"], result["test_code"],
                    )
                    continue
            except Exception as exc:
                logger.error("duplicate check failed for %s: %s", result.get("test_code"), exc)
                continue
```

CORE/E1381.PY

8 · The real wire: E1381

This is the layer a browser demo hides. A real analyzer will not send a message until it receives an ACK for its ENQ, and it repeats any frame whose checksum the receiver rejects. `E1381Framer` is incremental, so a frame split across TCP segments (or arriving slowly over 9600 baud) is assembled correctly, and stray noise before `STX` is resynced instead of corrupting the block.

```
def compute_checksum(frame_number: int, text: bytes) -> int:
    """Sum of frame number + text + ETX, low byte."""
    total = frame_number + sum(text) + ETX
    return total & 0xFF


def checksum_chars(value: int) -> bytes:
    """Checksum as the two ASCII nibbles the wire format uses."""
    return bytes(((value >> 4) + 0x30, (value & 0x0F) + 0x30))
```

```
    def feed(self, chunk: bytes) -> List[Event]:
        self.buffer.extend(chunk)
        events: List[Event] = []

        while self.buffer:
            head = self.buffer[0]

            # --- link control bytes ----------------------------------
            if head == ENQ:
                del self.buffer[0]
                events.append(Event("enq"))
                continue
            if head == EOT:
                del self.buffer[0]
                events.append(Event("eot"))
                continue
            if head in (ACK, NAK):
                del self.buffer[0]      # we are the receiver; ignore stray ACKs
                continue
            if head in (CR, LF, 0x00):
                del self.buffer[0]      # filler between blocks
                continue

            # --- a framed block --------------------------------------
            if head == STX:
                end = self.buffer.find(bytes([ETX]))
                if end == -1:
                    break               # wait for the rest
                if len(self.buffer) < end + 3:
                    break               # checksum bytes not here yet
                frame_number = (self.buffer[1] - 0x30) % 8   # 1..8 then 1 again
                payload = bytes(self.buffer[2:end])
                high, low = self.buffer[end + 1], self.buffer[end + 2]
                received = ((high - 0x30) << 4) | (low - 0x30)
                expected = compute_checksum(self.buffer[1], payload)
                ok = received == expected
                if not ok:
                    logger.warning(
                        "checksum mismatch on frame %d (got 0x%02X, expected 0x%02X)",
                        frame_number, received, expected,
```

LISTENERS/TCP_LISTENER.PY

9 · Listener framing

`_extract` turns a byte stream into messages: MLLP (`\x0b…\x1c`), ASTM (`\x05`/`\x03`), or bare text closed by the client. Whatever framing arrives, the analyzer gets `0x06` for success and `0x15` for a rejected message.

```
    def _extract(self, buffer: bytes):
        """Pull one MLLP message (VT ... FS CR) out of the stream buffer."""
        if not buffer:
            return None, buffer, False
        start = buffer.find(MLLP_START)
        if start == -1:
            return None, buffer[-1:], False        # keep only the tail
        end = buffer.find(MLLP_END, start)
        if end == -1:
            return None, buffer[start:], False
        payload = buffer[start + 1:end]
        remaining = buffer[end + 1:]
        if remaining.startswith(CR):
            remaining = remaining[1:]
        return payload, remaining, True
```

TESTS/TEST_PIPELINE.PY

10 · Proving it works

Fifty tests cover the parsers, the sniffer, the validator, de-duplication, the ACK modes and the error paths. No Oracle, no sockets, no lab instrument needed.

test run

```
python -m unittest discover -s tests -v
  ...
  Ran 50 tests in 0.068s
  OK
```

```
    def test_end_to_end_astm(self):
        response = self.engine.process(SAMPLES["astm_cbc_normal"], ANALYZER, peer="tcp:1")
        self.assertTrue(response["ok"])
        self.assertEqual(response["status"], "PARSED")
        self.assertEqual(response["stored"], 4)
        self.assertEqual(response["ack"], "\x06")
        # HGB was remapped to HB by the config map
        stored_codes = {r["test_code"] for r in self.store.results}
        self.assertIn("HB", stored_codes)
        self.assertEqual(self.store.stats()["total"], 4)

    def test_end_to_end_hl7(self):
```

TOOLS/DEVICE_SIM.PY, TOOLS/SNIFF_COM.PY

11 · Testing against a real machine

`device_sim.py` is a lab instrument simulator at the byte level: it performs the ENQ/ACK handshake, sends checksummed frames, honours NAK with a retry, and can inject the faults only a cable produces (bad checksum, line noise, dropped blocks). `sniff_com.py` captures the real analyzer's own bytes — over serial or as a TCP tap — and `replay_capture.py` replays that capture against the middleware, so the parser can be regression-tested with real machine data.

three levels of realism: simulate → capture → replay

```
python tools/device_sim.py -s astm_cbc_normal -p 6661 --trace
python tools/device_sim.py -s astm_cbc_normal --fault checksum --fault-rate 0.4
python tools/sniff_com.py --port COM4 --baud 9600 --parity EVEN \
       --tap 127.0.0.1:6661 --out logs/capture.jsonl
python tools/replay_capture.py logs/capture.jsonl --port 6661 --timing
```

DATABASE/CONTROL.PY,SERVICE.PY

12 · APEX-driven control plane

The lab must never edit a file or open a terminal. Configuration therefore lives in Oracle tables and is edited on APEX pages; `service.py` is a supervisor that polls those tables and applies the difference. `DatabaseControl` reads `lis_config` into a `ConfigSnapshot`, refuses anything invalid, claims `lis_commands` atomically and writes the heartbeat back. `FileControl` implements the same interface over `config.yaml`, so the same code path runs in the demo (`--mode file`).

```
    def config_version(self) -> str:
        """Cheap fingerprint; changes as soon as any row is edited.

        Only `config_version` (bumped by the DB trigger on a real edit) and the
        row count are used — NOT `applied_at`, which the service itself writes.
        Including it would make the service reload on every tick.
        """
        sql = f"""
            SELECT NVL(SUM(config_version), 0), NVL(MAX(config_version), 0), COUNT(*)
              FROM {self.schema}.lis_config
        """
        try:
            with self.db.cursor() as cur:
                cur.execute(sql)
                total, highest, count = cur.fetchone()
            return f"{total}|{highest}|{count}"
        except Exception as exc:
            logger.debug("config_version failed: %s", exc)
```

The fingerprint deliberately ignores `applied_at`: the service writes that column itself, and a fingerprint that included it would make the service reload on every single tick.

```
    async def _apply(self, snapshot: ConfigSnapshot) -> None:
        """Start/stop/restart only what actually changed."""
        wanted = {a.name: a for a in snapshot.analyzers if a.enabled}
        changed: List[str] = []

        # 1. stop listeners that disappeared or whose settings changed
        for name in list(self.listeners):
            listener = self.listeners[name]
            target = wanted.get(name)
            if target is None or target.as_dict() != getattr(listener, "_config", target.as_dict
    ()):
                await self._stop_listener(name)
                changed.append(name)

        # 2. build a fresh engine for the whole snapshot
        self.engine = self._build_engine(snapshot)

        # 3. start the missing ones
        loop = asyncio.get_running_loop()
        for name, analyzer in wanted.items():
            if name in self.listeners:
                continue
            if name in self._stopped_names:
                logger.info("%s stayed stopped (STOP command from APEX)", name)
                continue
            try:
                listener = await self._start_listener(analyzer.as_dict(), loop)
                self.listeners[name] = listener
                changed.append(name)
            except Exception as exc:
                logger.error("could not start %s: %s", name, exc)
                self.control.set_last_error(f"{name}: {exc}", name)

        self.snapshot = snapshot
        if changed:
            logger.info("listeners now: %s", ", ".join(sorted(self.listeners)) or "-")

    def _build_engine(self, snapshot: ConfigSnapshot) -> IngestEngine:
        settings = snapshot.settings
        known = None
        if settings.strict_test_codes:
```

Only listeners whose settings actually changed are stopped and restarted; unchanged ports keep their connections, so an operator editing one analyzer never drops another one's session. A listener that cannot bind its port reports the error instead of dying silently.

```
    async def _run_command(self, command: Dict[str, Any]) -> None:
        kind = command["command_type"]
        payload = command.get("payload")
        name = command.get("analyzer_name") or ""
        logger.info("command %s: %s %s", command["command_id"], kind, name)

        try:
            if kind == "PING":
                ok, message = True, f"alive, {len(self.listeners)} listener(s), uptime {self.upt
    ime()}"

            elif kind == "RELOAD":
                result = await self.reload(triggered_by=f"APEX command {command['command_id']}")
                ok, message = result["ok"], result["message"]

            elif kind in ("START", "STOP", "RESTART"):
                ok, message = await self._control_listener(kind, name, payload)

            elif kind == "SELFTEST":
                ok, message = await self._selftest(name, payload)

            else:
                ok, message = False, f"unknown command type {kind!r}"

        except Exception as exc:
            ok, message = False, f"{type(exc).__name__}: {exc}"
            logger.exception("command %s failed", command["command_id"])

        self.control.finish_command(command["command_id"], ok, message)
        self.control.insert_audit(f"CMD_{kind}", name or "-", message)
```

Commands come from the same poll: a `RELOAD`, `START`, `STOP`, `RESTART`, `SELFTEST` or `PING` row is claimed with `status = 'RUNNING'`, executed, and closed with `DONE`/`FAILED` plus a human-readable `result_message` that APEX shows next to the button.

Table	Written by	Read by
lis_config	APEX form	service (every 5 s)
lis_test_code_map	APEX form	service
lis_commands	APEX button	service (claims, then closes)
lis_analyzer_status	service (heartbeat)	APEX dashboard
lab_results / lis_raw_messages	service (pipeline)	APEX reports

> **Deployment**
> `ops/install_service_windows.ps1` (NSSM, auto-restart) and `ops/lis-middleware.service` (systemd) both run `service.py`, i.e. APEX mode with hot reload. `python main.py` stays the developer entry point.

> **Where to start reading**
> Follow the order of this document: main.py → engine.py → parsers/ → validator.py→ db_writer.py. Everything else (demo, tools) reuses those five files.